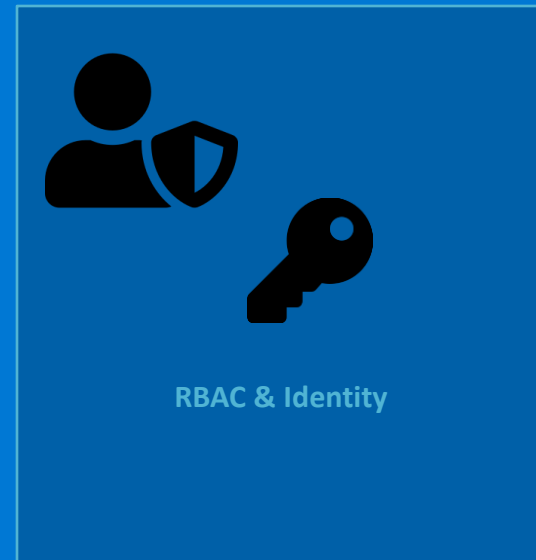


Gestion des identités et autorisations AKS

Azure AD, RBAC Kubernetes, ServiceAccounts et intégration IAM



- 1 Identités dans Kubernetes
- 2 Méthodes d'authentification
- 3 RBAC K8s — Roles et ClusterRoles
- 4 Intégration Azure AD (Entra ID)
- 5 Travaux pratiques

Identities dans Kubernetes

Deux types d'identités — utilisateurs humains et comptes de service (pods)



User / Group

Représentent des humains ou des systèmes externes.
Kubernetes ne gère pas les users — délégué à Azure AD sur AKS.
Authentification via certificat, token OIDC ou kubeconfig.

Ex : Développeur, DevOps, CI/CD pipeline



ServiceAccount

Identité pour les pods qui appellent l'API K8s.
Créé automatiquement (default) ou manuellement.
Token JWT monté automatiquement dans /var/run/secrets.

Ex : Prometheus, Flux, agent CI/CD dans un pod

Créer un ServiceAccount et un token

```
kubectl create serviceaccount myapp-sa -n production  
kubectl create token myapp-sa -n production --duration=1h # token JWT valide 1h
```

RBAC Kubernetes — Roles et ClusterRoles

Contrôler qui peut faire quoi sur quelles ressources

4 ressources RBAC à connaître

Role

Namespace

Permissions dans UN namespace

ClusterRole

Cluster

Permissions dans TOUT le cluster

RoleBinding

Namespace

Attache un Role à un User/SA

ClusterRoleBinding

Cluster

Attache un ClusterRole globalement

Exemple complet Role + RoleBinding

```
kind: Role # Lecture pods dans production
rules:
- apiGroups: ["" ] resources: [
  "pods" ]
  verbs: [
    "get", "list", "watch" ]
---
kind:
RoleBinding
subjects:
- kind:
User name: developer@company.com
roleRef.name: pod-reader
```

Azure RBAC sur AKS

Azure Kubernetes Service Cluster Admin

Accès total kubectl

AKS Cluster User Role

Récupérer kubeconfig

AKS RBAC Admin

RBAC K8s complet

AKS RBAC Reader

Lecture seule K8s

AKS RBAC Writer

Déployer sans RBAC admin

Intégration Azure AD (Entra ID) avec AKS

Authentifier les utilisateurs K8s via leur compte Azure



Activer AAD + Azure RBAC

```
# Activer Azure AD + RBAC sur AKS existant
az aks update -g myRG -n myAKS \
  --enable-azure-rbac \
  --enable-aad

# Assigner un rôle Azure à un utilisateur
AKS_ID=$(az aks show -g myRG -n myAKS --query id -o tsv)
az role assignment create \
  --role 'Azure Kubernetes Service RBAC Reader' \
  --assignee user@company.com \
  --scope $AKS_ID

# Récupérer les credentials
az aks get-credentials -g myRG -n myAKS
kubectl get nodes # Demande login AAD
```

Bonnes pratiques AAD + AKS



Utiliser des groupes AAD plutôt que des utilisateurs individuels dans les RoleBindings



Activer MFA (Multi-Factor Auth) sur les comptes ayant accès au cluster



Les tokens OIDC expirent — kubectl re-authentifie automatiquement



Principe du moindre privilège — donner uniquement les droits nécessaires



Activer les logs d'audit K8s pour tracer tous les accès

TP — RBAC : accès limité à un namespace

Durée : 30 min | Prérequis : cluster AKS avec Azure AD activé, kubectl admin

1

Créer un namespace et un ServiceAccount

```
kubectl create ns team-alpha  
kubectl create serviceaccount alpha-dev -n team-alpha
```

✓ Validation :

Namespace et ServiceAccount créés

2

Créer un Role lecture seule

```
kubectl apply -f role-reader.yaml  
# verbs: get,list,watch resources: pods,deployments,services  
kubectl get role -n team-alpha
```

✓ Validation :

Role pod-reader visible dans team-alpha

3

Binder le Role au ServiceAccount

```
kubectl create rolebinding alpha-binding \  
--role=pod-reader --serviceaccount=team-alpha:alpha-dev -n team-alpha
```

✓ Validation :

RoleBinding créé dans team-alpha

4

Tester les permissions

```
kubectl auth can-i get pods -n team-alpha --as=system:serviceaccount:team-alpha:alpha-dev  
kubectl auth can-i delete pods -n team-alpha --as=system:serviceaccount:team-alpha:alpha-dev  
# get: yes delete: no
```

✓ Validation :

Lecture autorisée — suppression refusée ✓